X X X X X

X X X X X

X

Abstract. PCR amplification during Illumina library preparation can introduce chimeric reads that degrade mitochondrial genome assembly by creating spurious junctions and fragmented assembly graphs. Existing chimera detection tools are largely optimized for amplicon-based microbial workflows and often rely on reference databases or abundance assumptions that do not fit single-species organellar sequencing. **MitoChime** is presented as a pipeline for detecting PCR-induced chimeric reads in *Sardinella lemuru* mitochondrial paired-end data using alignment-derived features and a sequence-based convolutional model. Using simulated clean and chimeric reads, alignment-structure and clipping metrics are extracted alongside k-mer discontinuity and microhomology descriptors, and supervised classifiers are benchmarked against a CNN. On a held-out test set, a tuned Gradient Boosting classifier achieves F1=0.776 and ROC–AUC=0.846 for chimeric-read detection, while a CNN1D model achieves F1=0.908 and ROC–AUC=0.958 with higher recall. Assembly evaluation using SPAdes-generated assembly graphs summarized with Bandage statistics shows that filtering reduced assembly graph complexity at high chimera rates, with performance depending on read depth. These results support MitoChime as a practical pre-assembly quality-control step for mitochondrial genome assembly pipelines.

Keywords: Chimera detection · Mitochondrial genome assembly · Illumina reads · Machine learning · Deep learning

1 Introduction

Polymerase chain reaction (PCR) amplification during next-generation sequencing (NGS) library preparation can generate chimeric reads, which are artificial recombinant sequences formed when partially copied DNA fragments attach to different templates and are extended in later cycles [8, 10]. Despite the high throughput and accuracy of Illumina sequencing, PCR-induced chimeras remain a persistent artifact that can affect downstream analyses and compromise genome assembly.

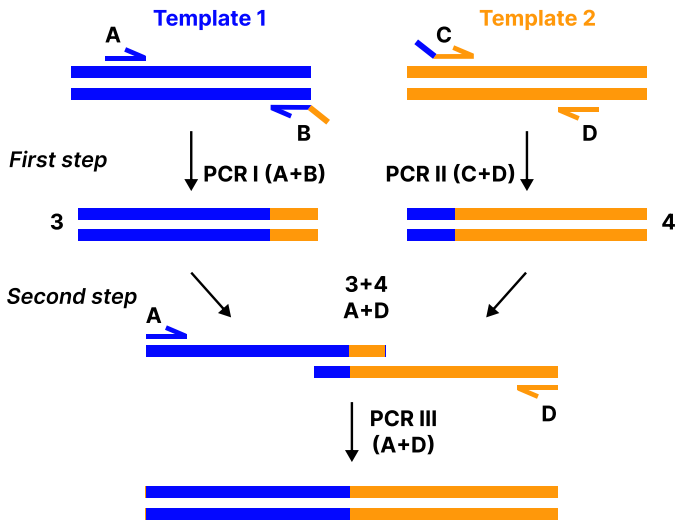

Fig. 1: Schematic illustration of PCR-induced chimera formation through incomplete extension and template switching, adapted from [13].

Mitochondrial genomes are particularly susceptible to this issue because they are small, circular, and often contain repetitive regions [1, 2]. Under some conditions, junction-supporting artifacts may introduce misleading connections in assembly graphs, produce branching structures or fragmentation, or in some cases contribute to assembly failure [3, 6, 7]. Because many organelle assembly tools such as GetOrganelle [7] and MITObim [6] assume that input reads are largely free of such artifacts, chimera-related assembly failure modes remain insufficiently addressed.

Many existing chimera detection approaches were originally developed for amplicon-based microbial community sequencing and often use differences in sequence frequency to help identify chimeras [5, 12]. These assumptions are not well aligned with single-species organellar sequencing, highlighting the need for a more specialized approach. A pipeline that detects chimeric reads from alignment and sequence irregularities can therefore be used as a pre-assembly filtering step.

This study introduces MitoChime, a pipeline for detecting PCR-induced chimeric reads in *Sardinella lemuru* mitochondrial Illumina paired-end data. The pipeline extracts interpretable features from standard BAM and FASTQ artifacts, benchmarks supervised machine learning models together with a sequence-based convolutional neural network (CNN), and evaluates the effects of filtering on downstream assembly graph complexity.

Contributions. The study makes three main contributions. First, it develops a read-level feature set that combines alignment structure, clipping, k-mer dis-

continuity, and microhomology descriptors relevant to PCR-induced chimerism. Second, it benchmarks and tunes multiple classifiers while analyzing feature contributions to identify the most informative predictors. Third, it evaluates how filtering affects assembly graph complexity across different read depths and chimera rates to assess its benefits and limitations.

2 Related Work

Chimera detection in NGS has been widely studied in amplicon sequencing, where tools such as UCHIME and UCHIME2 detect sequences that appear to be recombined from two parent sequences [5, 4]. Although effective in many microbial studies, these approaches are not designed for single-species organellar sequencing data.

Ensemble approaches such as CATCh use the scores and binary decisions produced by multiple chimera detection tools as input to a supervised learning model, improving detection in 16S rRNA amplicon sequencing but still relying on the sequence and alignment signals examined by those tools [9]. Pipelines such as ChimPipe detect biological chimeras in RNA-seq using discordant paired-end and split-read evidence, but are intended for fusion detection and usually require reference genome and annotation data [11].

In contrast, mitochondrial assembly pipelines generally do not provide explicit read-level chimera filtering tailored to PCR artifacts. This gap motivates a feature-driven approach that targets alignment fragmentation and sequence discontinuity in mitochondrial paired-end reads and evaluates success in terms of downstream assembly graph outcomes.

3 Methods

3.1 Overview

Figure 2 summarizes the MitoChime workflow: paired-end read simulation, alignment to a mitochondrial reference, (i) tabular feature extraction for classical machine learning and (ii) sequence-only dataset preparation for deep learning, followed by model inference, pair-safe read filtering, and downstream assembly evaluation.

3.2 Data simulation and alignment

Paired-end Illumina reads were simulated from the *Sardinella lemuru* mitochondrial reference (NC_039553.1). Clean read pairs were generated with `wgsim` (150 bp) using an error rate of 0.05. Chimeric templates were constructed by concatenating two non-adjacent reference segments, optionally retaining short overlaps to approximate PCR template switching, and were simulated with the same read-length configuration but with error rate set to 0 to preserve junction signals. Reads were aligned to the reference using `minimap2` in short-read mode (`-ax sr`) and processed with `samtools` to produce sorted and indexed BAM files.

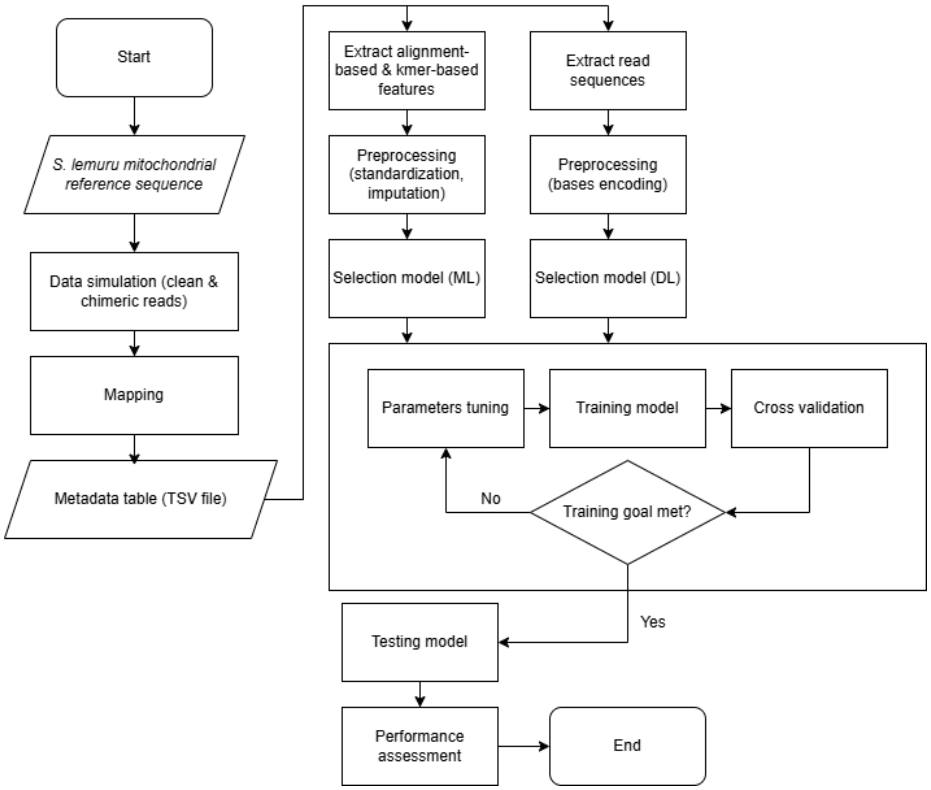

Fig. 2: MitoChime workflow from simulated reads to filtering and assembly evaluation.

3.3 Feature and sequence extraction

Two complementary input representations were prepared from the same labeled read pairs to support both classical machine learning and deep learning.

Tabular feature matrix (classical machine learning). For each primary-mapped read, a fixed-length feature vector was computed using `pysam` and NumPy. Feature families included: (i) supplementary-alignment and alignment-structure summaries derived from SA tags (e.g., presence/count of supplementary alignments, segment distances, strand consistency, and MAPQ/NM aggregates); (ii) CIGAR-derived clipping statistics (e.g., left/right soft clipping, total clipped bases, and an inferred breakpoint position); (iii) k-mer composition discontinuity between read segments ($k = 6$), quantified via cosine distance and Jensen–Shannon divergence between left/right k-mer profiles; and (iv) microhomology descriptors around the inferred breakpoint (longest exact suffix–prefix overlap within a ± 40 bp window and its GC fraction), together with basic alignment-quality indicators.

Sequence-only dataset (deep learning). In parallel, the raw nucleotide sequence (length $L = 150$) was retained as a separate input for the CNN. Sequences were one-hot encoded at training and inference time, with four base channels (A, C, G, T). Ambiguous bases were represented as all zeros.

3.4 Modeling and evaluation protocol

The labeled dataset was partitioned into training and test splits (80/20) using stratification to preserve the balanced class ratio. Model selection and tuning were performed on the training split only. Final performance was reported on the held-out test split using accuracy, precision, recall, F1, and ROC–AUC (precision/recall/F1 for the chimeric class).

Classical machine learning (feature-based). Feature-based models were implemented in Python using `scikit-learn`, `xgboost`, `lightgbm`, and `catboost`. An initial benchmark evaluated a panel of classifiers under five-fold stratified cross-validation on the training split, including a dummy baseline, L2-regularized logistic regression, a calibrated linear SVM, k-nearest neighbours, Gaussian Naïve Bayes, tree ensembles (Random Forest, ExtraTrees, and bagging with decision trees), and gradient boosting methods (Gradient Boosting, XGBoost, LightGBM, and CatBoost). Based on cross-validation ranking, a subset of competitive model families was carried forward for hyperparameter optimization. Hyperparameters were tuned using randomized search with five-fold stratified cross-validation (`RandomizedSearchCV`), optimizing chimeric-class F1. Among the feature-based models, a tuned `GradientBoostingClassifier` (`sklearn.ensemble`) achieved the best validation ranking and was used as the primary classical model. The hyperparameter search included the number of estimators, `learning_rate` (selected value 0.2), and tree-structure constraints; the selected model was refit on the full training split and evaluated once on the held-out test split.

Deep learning (sequence-based). A one-dimensional convolutional neural network (CNN1D) was trained using sequence-only input of length $L = 150$, represented by one-hot encoding. The architecture comprised three convolutional blocks with 64, 128, and 256 filters (kernel sizes 7, 5, and 3; ReLU activations; max-pooling after the first two blocks), followed by adaptive max pooling and a two-layer MLP head ($256 \to 128 \to 2$) with dropout (0.2). Training used mini-batches of 128 for up to 30 epochs with learning rate 0.001 and weight decay 10^{-4}. The selected checkpoint was the model achieving the best held-out F1 during training.

3.5 Filtering strategy

Filtering was performed using model scores for the chimeric class and a user-specified decision threshold $\tau \in [0, 1]$. For each read pair, the predicted chimeric probability p was computed and pairs were retained if $p < \tau$ (i.e., higher τ yields stricter chimera removal). Pair-safe filtering was enforced by normalizing

identifiers to a base read-pair ID: leading `@` characters and trailing mate suffixes (`/1`, `/2`) were removed, and unique base IDs were expanded to mate-specific identifiers by writing both `base/1` and `base/2`. Filtered FASTQ outputs were then produced by retaining only R1 records whose base IDs passed the filter and writing only those R2 records with matching base IDs, ensuring consistent retention/removal of both mates.

3.6 Assembly and downstream evaluation

Assemblies were generated from unfiltered and filtered read sets using SPAdes. Assembly graphs were summarized using Bandage graph statistics. Graph complexity was quantified using graph-level metrics including node and edge counts, together with total assembled length, across simulated read depths and chimera rates.

3.7 CLI workflow

A command-line interface automates preprocessing, model inference, pair-safe filtering, and optional assembly evaluation (Fig. 3) to support reproducible use in mitochondrial assembly workflows.

4 Discussion

Results show that PCR-induced chimeras in short-read mitochondrial data can be detected using lightweight, interpretable features derived from standard alignments, and that a sequence-based CNN can further improve sensitivity. The Gradient Boosting model is conservative, achieving high precision for the chimeric class but missing more chimeric reads. The CNN1D substantially improves recall and ROC–AUC, but increases false positives, which can reduce read support when coverage is low.

Downstream assembly evaluation indicates that optimal filtering depends on both coverage and chimera burden. At moderate-to-high coverage and high chimera rates, stronger filtering can simplify graphs and restore near-contiguity. At minimal depth, however, aggressive filtering may remove critical read support and introduce fragmentation. These findings suggest that deployment should expose a tunable threshold and report read-retention diagnostics to support user calibration.

Limitations include reliance on simulation and a single target species, and the use of a single alignment strategy for feature extraction. Broader validation across taxa, library preparation conditions, and empirical datasets is required to establish generalizability. Future work includes calibration on real sequencing libraries, evaluation under varying sequencing error profiles, and extension of the CLI workflow to support additional model families and references.

5 Conclusion

MitoChime provides a practical pre-assembly chimera detection pipeline for mitochondrial Illumina paired-end reads using interpretable alignment- and sequence-derived signals. A tuned Gradient Boosting classifier achieves robust performance with conservative filtering, while a CNN1D model achieves higher sensitivity and discrimination. External assembly evaluation demonstrates that filtering can substantially reduce assembly graph complexity under high chimera burdens, with coverage-dependent trade-offs at low depth. These results support MitoChime as a quality-control step that can improve mitochondrial reconstruction workflows in resource-constrained settings.

References

1. Boore, J.L.: Animal mitochondrial genomes. Nucleic Acids Research **27**(8), 1767–1780 (1999). https://doi.org/10.1093/nar/27.8.1767
2. Cameron, S.L.: Insect mitochondrial genomics: Implications for evolution and phylogeny. Annual Review of Entomology **59**, 95–117 (2014). https://doi.org/10.1146/annurev-ento-011613-162007
3. Dierckxsens, N., Mardulyn, P., Smits, G.: NOVOPlasty: de novo assembly of organelle genomes from whole genome data. Nucleic Acids Research **45**(4), e18 (2017). https://doi.org/10.1093/nar/gkw955
4. Edgar, R.C.: UCHIME2: improved chimera prediction for amplicon sequencing. bioRxiv (2016), `https://api.semanticscholar.org/CorpusID:88955007`
5. Edgar, R.C., Haas, B.J., Clemente, J.C., Quince, C., Knight, R.: UCHIME improves sensitivity and speed of chimera detection. Bioinformatics **27**(16), 2194–2200 (2011). https://doi.org/10.1093/bioinformatics/btr381
6. Hahn, C., Bachmann, L., Chevreux, B.: Reconstructing mitochondrial genomes directly from genomic next-generation sequencing reads—a baiting and iterative mapping approach. Nucleic Acids Research **41**(13), e129 (2013). https://doi.org/10.1093/nar/gkt371
7. Jin, J.J., Yu, W.B., Yang, J., Song, Y., dePamphilis, C.W., Yi, T.S., Li, D.Z.: GetOrganelle: a fast and versatile toolkit for accurate de novo assembly of organelle genomes. Genome Biology **21**(1), 241 (2020). https://doi.org/10.1186/s13059-020-02154-5
8. Judo, M.S.B., Wedel, W.R., Wilson, B.H.: Stimulation and suppression of PCR-mediated recombination. Nucleic Acids Research **26**(7), 1819–1825 (1998). https://doi.org/10.1093/nar/26.7.1819
9. Mysara, M., Saeys, Y., Leys, N., Raes, J., Monsieurs, P.: CATCh, an ensemble classifier for chimera detection in 16s rrna sequencing studies. Applied and Environmental Microbiology **81**(5), 1573–1584 (2015). https://doi.org/10.1128/AEM.02896-14, `https://journals.asm.org/doi/abs/10.1128/AEM.02896-14`
10. Qiu, X., Wu, L., Huang, H., McDonel, P.E., Palumbo, A.V., Tiedje, J.M., Zhou, J.: Evaluation of PCR-generated chimeras, mutations, and heteroduplexes with 16s rrna gene-based cloning. Applied and Environmental Microbiology **67**(2), 880–887 (2001). https://doi.org/10.1128/AEM.67.2.880-887.2001

11. Rodriguez-Martin, B., Palumbo, E., Marco-Sola, S., Griebel, T., Ribeca, P., Alonso, G., Djebali, S.: ChimPipe: Accurate detection of fusion genes and transcription-induced chimeras from RNA-seq data. BMC Genomics **18** (2017). https://doi.org/10.1186/s12864-016-3404-9
12. Rognes, T., Flouri, T., Nichols, B., Quince, C., Mahé, F.: VSEARCH: a versatile open source tool for metagenomics. PeerJ **4**, e2584 (2016). https://doi.org/10.7717/peerj.2584
13. Sieber, F.: Development of a Tool to Address Nucleic Acids into Mitochondria. Ph.D. thesis, Nantes Université (Sep 2011), phD thesis, advisor: Laurence Maréchal-Drouard

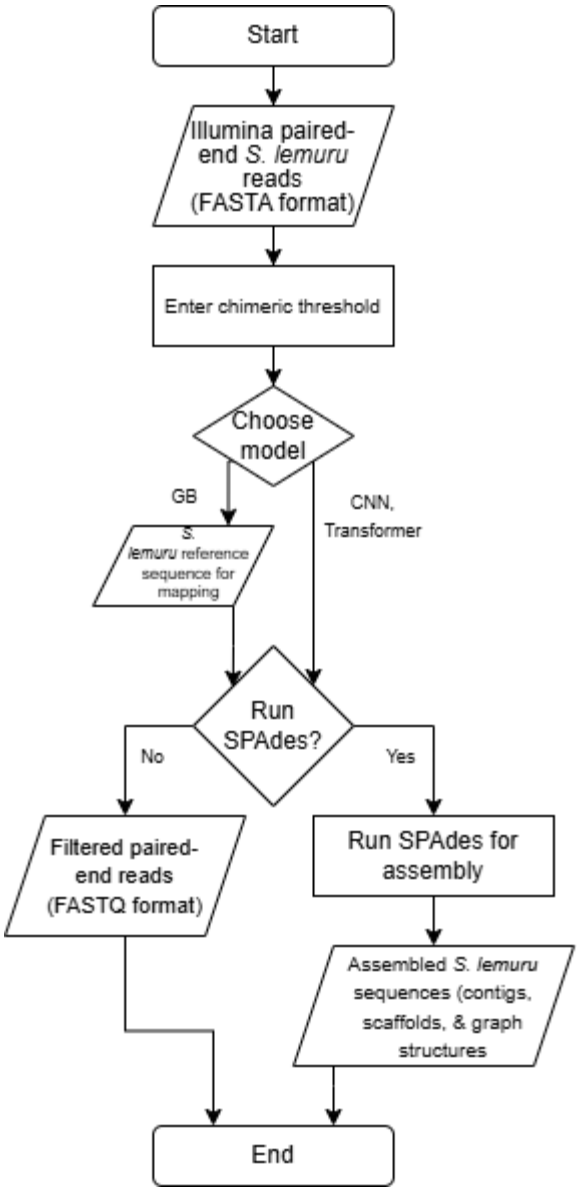

Fig. 3: MitoChime CLI workflow for model selection, filtering, and optional assembly evaluation.